

Enterprise MLOps End-to-End on GCP

UI-Only Lab Workshop — Visual Walkthrough

Business problem: Customer Churn Prediction. This companion visual guide pairs every major step of the lab with an annotated diagram of the Console screen you'll be using, so you can see roughly where to click before you get there.

Important note on the diagrams: these are schematic wireframes drawn to match the layout and wording of the Google Cloud Console (top bar, left navigation, main panel), not literal screenshots of the live product — Google updates its Console UI regularly, and exact colors/spacing may differ slightly on the day you run the lab. Numbered red circles correspond to the numbered instructions beside each image. Always follow the on-screen labels in your real Console as the source of truth.

Constraints respected in this lab:

- Region: asia-south1 (Mumbai) | Python 3.11 | CPU-only
- Model: Logistic Regression or Decision Tree, .joblib, ≤5 MB
- Historical dataset ≤25 MB, ≤5,000 rows, ≤15 features
- Cloud Run: 1 vCPU, 2 GB RAM

SECTION 1 — GCP Environment Setup and ML Deployment

Step 1.1–1.2 — Create the Project & Link Billing

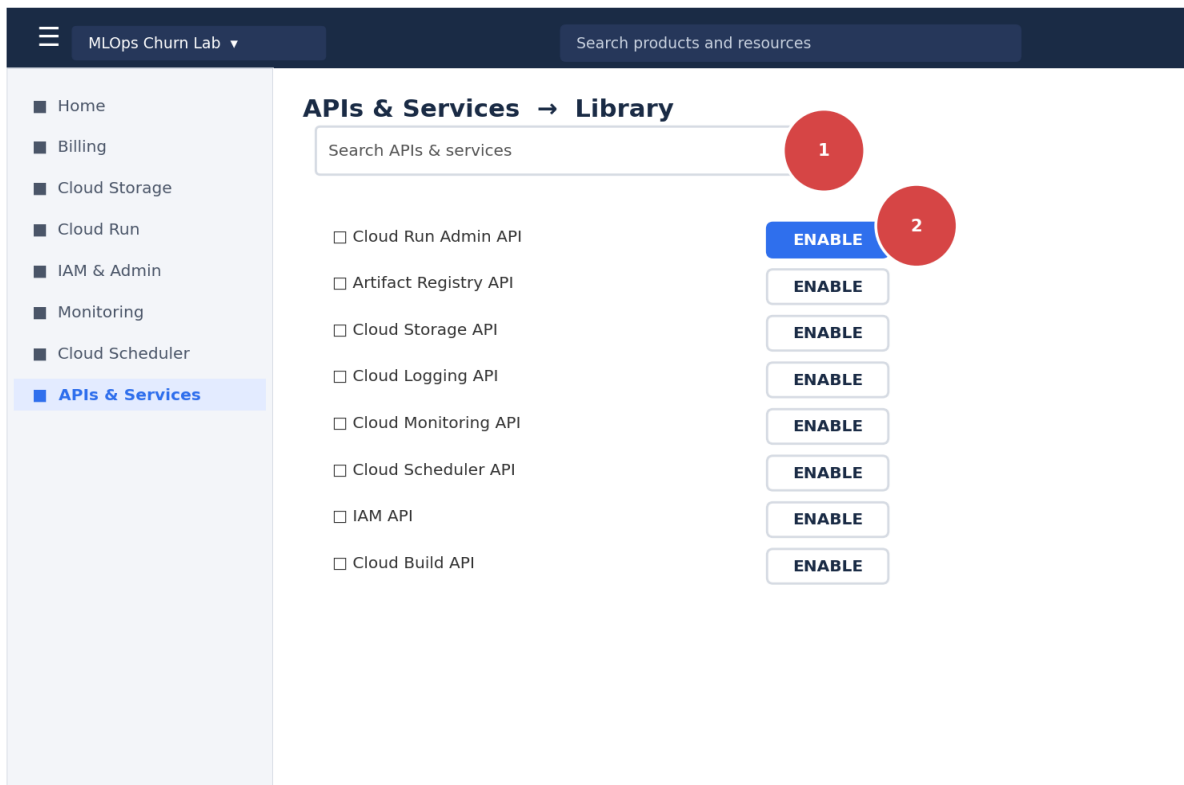
- 1. Click the project dropdown (top left) → New Project.
- 2. Name it "MLOps Churn Lab"; note the generated Project ID.
- 3. Click Create, then select it from the dropdown once ready. Then go to Billing → link a billing account to this project.

Step 1.2 (cont.) — Set a Budget & Alerts

The screenshot shows the Google Cloud console interface. On the left is a sidebar with a menu containing: Home, Billing (highlighted), Cloud Storage, Cloud Run, IAM & Admin, Monitoring, Cloud Scheduler, and APIs & Services. The top header has a hamburger menu icon, the text 'MLOps Churn Lab', and a search bar labeled 'Search products and resources'. The main content area is titled 'Billing → Budgets & alerts → Create Budget'. It contains a form with the following fields: 'Name: Churn Lab Budget' (marked with a red circle 1), 'Scope: This project only' (marked with a red circle 2), and 'Target amount: ₹500 / \$10' (marked with a red circle 3). Below these is an 'Alert thresholds' section (marked with a red circle 4) showing '50% • 90% • 100% of budget'. At the bottom is a blue 'FINISH' button (marked with a red circle 5).

- 1. Go to Billing → Budgets & alerts → Create Budget.
- 2. Name it "Churn Lab Budget", scope to this project only.
- 3. Set an amount (e.g. Rs.500 / \$10).
- 4. Set alert thresholds at 50%, 90%, 100%.
- 5. Click Finish.

Step 1.3 — Enable Required APIs



- 1. Go to APIs & Services → Library.
- 2. Search for each required API (Cloud Run Admin, Artifact Registry, Cloud Storage, Logging, Monitoring, Scheduler, IAM, Cloud Build) and click Enable on each one.

Step 1.5 — Create the Cloud Storage Bucket and Folders

- 1. Go to Cloud Storage → Buckets → Create.
- 2. Name it (globally unique), region = asia-south1 (Mumbai).
- 3. Storage class Standard, access control Uniform, click Create.
- 4. Inside the bucket, use Create folder to add: training-data, prediction-data, models, predictions, logs, artifacts.

Step 1.10 — Deploy to Cloud Run (Continuous Deploy from GitHub)

Cloud Run → Create Service

() Deploy one revision from an existing container image

(●) Continuously deploy new revisions from a repository

SET UP WITH CLOUD BUILD

Repository provider: GitHub → Authenticate

Repository: churn-mlops-lab Branch: main

Build type: Dockerfile (auto-detected)

Service name: churn-api Region: asia-south1

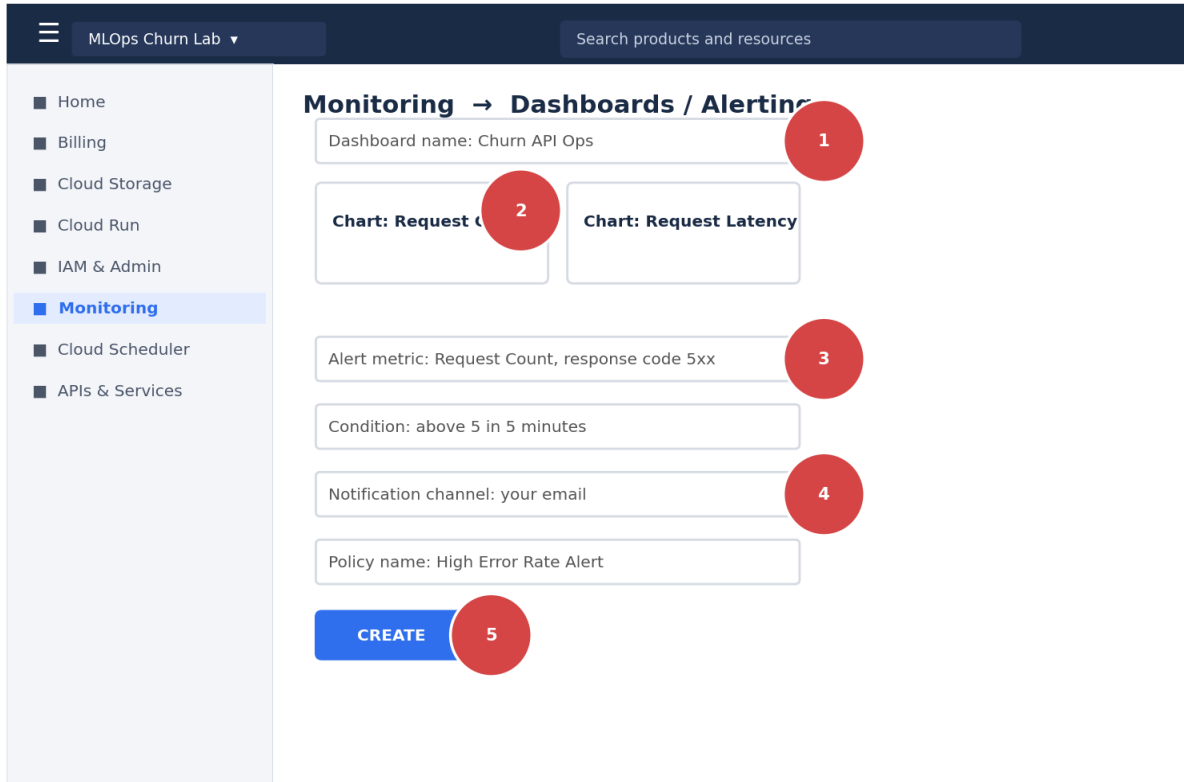
CREATE

- 1. Go to Cloud Run → Create Service.
- 2. Choose "Continuously deploy new revisions from a source repository".
- 3. Click Set up with Cloud Build, connect and authenticate GitHub.
- 4. Select repo churn-mlops-lab, branch main; Dockerfile is auto-detected.
- 5. Set service name churn-api, region asia-south1, allow unauthenticated invocations.
- 6. Click Create — Cloud Build will build & deploy automatically.

This is the step that replaces every gcloud/docker CLI command with a Console wizard.

SECTION 2 — Production Monitoring, Drift Detection, Experiment Tracking

Step 2.1 — Cloud Monitoring Dashboard & Alert Policy



- 1. Go to Monitoring → Dashboards → Create Dashboard, name it "Churn API Ops".
- 2. Add charts for Request Count and Request Latency (resource: Cloud Run Revision).
- 3. Go to Monitoring → Alerting → Create Policy.
- 4. Metric = Request Count, filtered to 5xx responses; threshold >5 in 5 minutes.
- 5. Add your email as the notification channel, name the policy, click Create.

Step 2.2–2.3 — Drift Detection Workflow (run locally, review in Console)

Generate a drifted batch locally (*generate_drift_data.py*), run Evidently's DataDriftPreset comparing **train** → **current (healthy)** and **train** → **drifted**, then upload both resulting HTML reports to **Cloud Storage** → **logs/** using the Console's Upload files button, and open each from the file's  menu to compare them side by side.

Tip: this reuses the same bucket UI shown in Step 1.5 above — just navigate to the logs/ folder instead of training-data/.

Step 2.4 — Experiment Tracking with MLflow (local web UI)

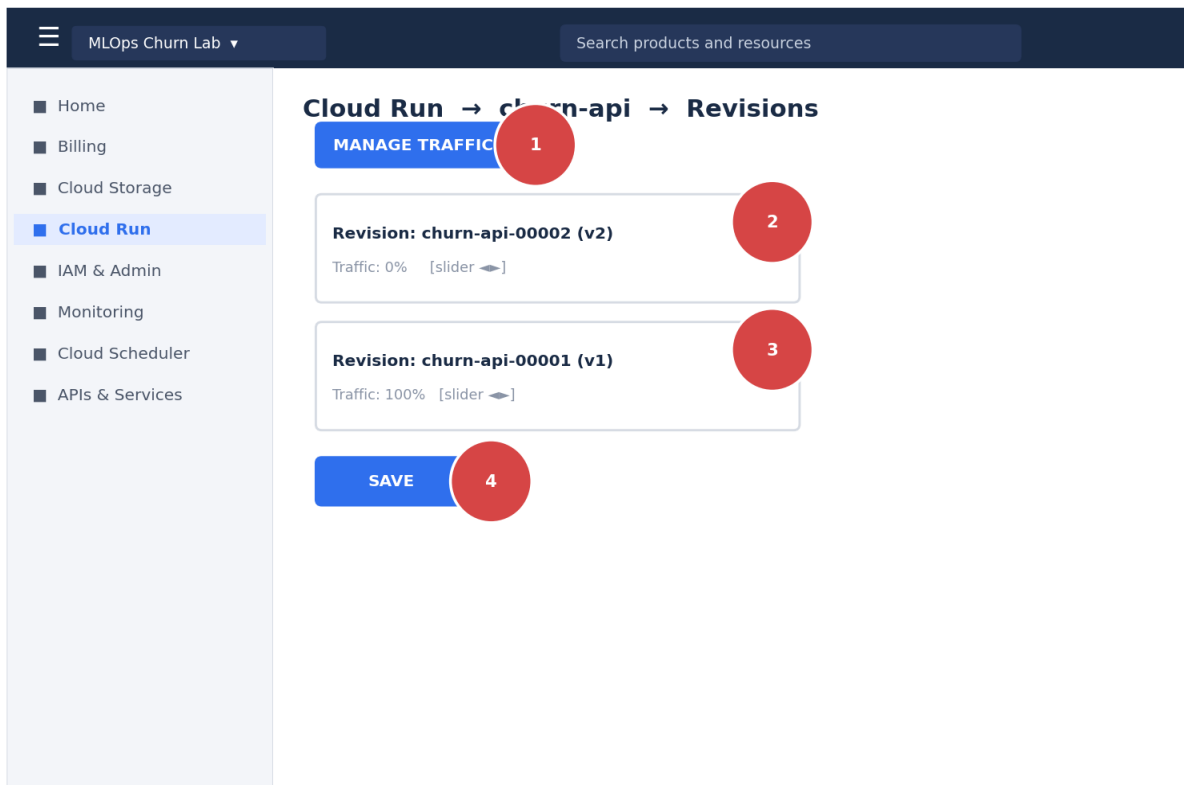
Run *track_experiment.py* locally, then run `mlflow ui --port 5000` and open `http://localhost:5000` in a browser. This is MLflow's own web interface — click into each run to show logged parameters, metrics, and the model artifact.

SECTION 3 — Retraining, Model Lifecycle, and Governance

Step 3.1–3.2 — Retrain, Compare, and Redeploy v2

Run `retrain_model.py` locally to train a challenger model and compare its F1 against the production model on the same validation split; both runs are logged to MLflow. If the challenger wins, upload the new `churn_model.joblib` to Cloud Storage (models/ folder — same Upload files flow as before) and commit it to GitHub; Cloud Run rebuilds and redeploys automatically.

Step 3.3 — Review Revisions and Roll Back



- 1. Go to Cloud Run → churn-api → Revisions tab.
- 2. Click Manage Traffic.
- 3. Set 100% traffic to the older (v1) revision, 0% to v2.
- 4. Click Save, then confirm via `GET /metadata` that it now reports v1 again.

The same screen supports splitting traffic (e.g. 90/10) for canary deployments.

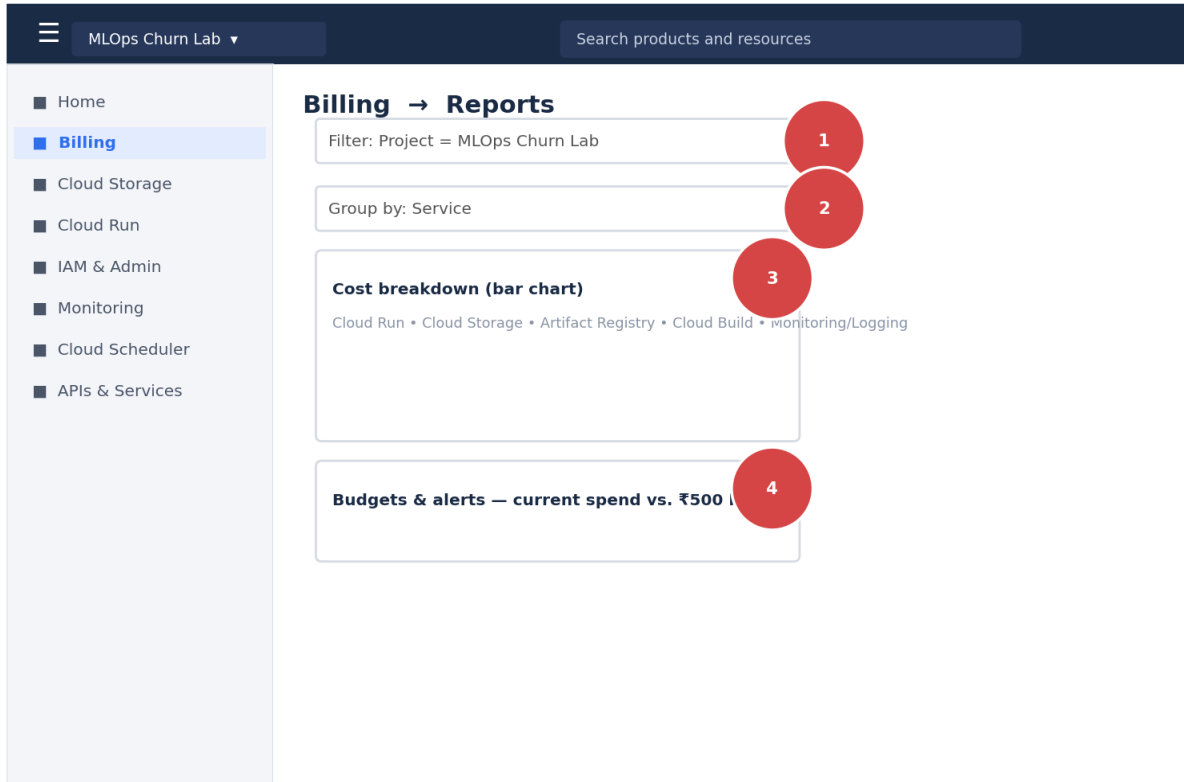
Step 3.4 — Governance: Service Accounts & IAM Review

The screenshot shows the Google Cloud IAM & Admin console. The left sidebar contains a navigation menu with options: Home, Billing, Cloud Storage, Cloud Run, IAM & Admin (highlighted), Monitoring, Cloud Scheduler, and APIs & Services. The main content area is titled 'IAM & Admin → Service Accounts'. Below the title is a blue button labeled 'CREATE SERVICE ACCOUNT' (1). Below this are two input fields: 'Name: churn-viewer' (2) and 'Role: Cloud Run Viewer' (3). Below the input fields are two blue buttons: 'CONTINUE' and 'DONE' (4). Below the buttons is a section titled 'Audit Logs' (5) with a filter set to 'Cloud Run Admin API — view deploy history'.

- 1. Go to IAM & Admin → Service Accounts → Create Service Account.
- 2. Name it "churn-viewer".
- 3. Assign role Cloud Run Viewer (read-only — cannot deploy or delete).
- 4. Click Continue → Done.
- 5. Review Audit Logs filtered to Cloud Run Admin API to see every deploy action.

SECTION 4 — Cost, ROI, Automation, and Architecture

Step 4.1–4.2 — Review Costs in Billing Reports



- 1. Go to Billing → Reports.
- 2. Filter by Project = MLOps Churn Lab.
- 3. Group by Service to see the cost breakdown across Cloud Run, Storage, Artifact Registry, Cloud Build, Monitoring/Logging.
- 4. Cross-check current spend against the budget created in Step 1.2.

Pair this screen with the `cost_and_roi.py` script output to connect infra spend to business ROI (revenue saved, break-even customers).

Step 4.3 — Automate with Cloud Scheduler

- 1. Go to Cloud Scheduler → Create Job.
- 2. Name churn-daily-predictions, region asia-south1.
- 3. Cron frequency 0 6 * * *, timezone Asia/Kolkata.
- 4. Target: HTTP POST to your-service-url/run-batch-predict.
- 5. Add an OIDC auth token if the endpoint requires authentication, click Create.
- 6. Use the  menu → Force run to test immediately without waiting for 6 AM.

Step 4.4 — Full Architecture Recap

Business Problem → Historical Data → Model (Logistic Regression)
→ Cloud Storage (training-data/, models/)
→ FastAPI → Docker → GitHub → Cloud Build → Cloud Run
→ Predictions → Cloud Storage (predictions/)
→ Monitoring (Logging + Monitoring) → Drift Detection (Evidently)
→ Experiment Tracking (MLflow) → Retraining → Model Versioning (v1 → v2)
→ Governance (IAM, audit logs) → Automation (Cloud Scheduler)
→ Business Evaluation (Cost + ROI)

SECTION 5 — Conclusion, Cleanup, and Q&A;

Close the workshop with mandatory cleanup — every step below is done in the Console, the same way each resource was created:

- Cloud Scheduler → churn-daily-predictions → Delete
- Cloud Run → churn-api → Delete
- Cloud Build → Triggers → delete the auto-created GitHub trigger
- Artifact Registry → delete the repository Cloud Build created
- Cloud Storage → your-bucket-name → Delete
- Monitoring → delete the alert policy and dashboard
- Optional: IAM & Admin → Settings → Shut down project

Closing message: enterprise MLOps is a continuous lifecycle — technical performance, operational reliability, governance, cost, and business value all have to be managed together, and every one of those pieces was demonstrated using nothing but the Google Cloud Console.